

Paper II Audit -- Gate H: Independent Lean verification

Test	Gate H -- Independent Lean verification
Target	PaperII.theorem_1_2 (unconditional, IsChordal)
Toolchain	leanprover/lean4:v4.28.0 / lake 5.0.0
Build result	(see lake_build.log)

1. What was done

(1) lake build PaperII in the project lean/ tree; (2) an auditor-authored AuditCheck.lean running #print axioms PaperII.theorem_1_2, #check of the statement, and #print of IsChordal; (3) a SHA-256 manifest of the exact Lean source built (lean_source_sha256.txt).

2. Build result

(see lake_build.log). The build log contains no error/failed/sorry line (only benign unusedSectionVars linter warnings).

3. Statement shape (read + checked)

PaperII.theorem_1_2 (Unconditional.lean:72-79) has hypotheses exactly $\text{Fintype.card } W = n$ and $\text{IsChordal } G$ -- NO ChordalStructure, NO A2Transfer -- and asserts BOTH the forall upper bound $\phi\tau G \leq \text{floor}((2n+1)^2/24)$ AND the exists attainment. $\phi\tau$ (Model.lean:63) = $\text{edgeFinset.card} - 2*\tau_{3\text{star}}$. IsChordal (Chordal.lean:35) is the standard chord definition (every cycle of length ≥ 4 has a chord); not weakened.

4. #print axioms output

```
'PaperII.theorem_1_2' depends on axioms: [propext, Classical.choice, Quot.sound]
theorem_1_2 :  $\forall (n : \mathbb{N}),$ 
   $1 \leq n \rightarrow$ 
    ( $\forall \{W : \text{Type}\} [\text{inst} : \text{Fintype } W] [\text{inst}_1 : \text{DecidableEq } W] (G : \text{SimpleGraph } W) [\text{inst}_2 : \text{DecidableRel } G.\text{Adj}],$ 
       $\text{Fintype.card } W = n \rightarrow \text{IsChordal } G \rightarrow \phi\tau G \leq \uparrow((2 * \uparrow n + 1) ^ 2 / 24)) \wedge$ 
       $\exists W \times x_1 G x_2, \text{Fintype.card } W = n \wedge \text{IsChordal } G \wedge \phi\tau G = \uparrow((2 * \uparrow n + 1) ^ 2 / 24)$ 
    )
@IsChordal :  $\{V : \text{Type } u_1\} \rightarrow \text{SimpleGraph } V \rightarrow \text{Prop}$ 
def PaperII.IsChordal.{u_1} :  $\{V : \text{Type } u_1\} \rightarrow \text{SimpleGraph } V \rightarrow \text{Prop} :=$ 
fun {V} G =>
   $\forall c : G.\text{Walk } v v,$ 
     $c.\text{IsCycle} \rightarrow 4 \leq c.\text{length} \rightarrow \exists x y, x \in c.\text{support} \wedge y \in c.\text{support} \wedge G.\text{Adj } x y \wedge s(x, y) \notin c.\text{edges}$ 
```

5. Sorry-free

No sorry/admit tactic occurs in lean/PaperII/*.lean; the only 'sorry' substrings are the words 'sorry-free' in docstrings (Dirac1.lean:9, FiniteCover.lean:11).

6. Residual (honest)

Mathlib came from the project's pinned lake-manifest cache, not a clean-room from-scratch rebuild. The SHA-256 match to commit 5f2d448 could not be verified here (git history unreachable; release package lacks the lean/ tree). lean_source_sha256.txt records exactly what was built.